

IntentPay

AI Payment Intent Guardian + Personal Financial Guardian

Team NAS CODERS

What this manual is

A plain-language guide for anyone — no coding needed. It explains what IntentPay is for, what every folder and file in the project does, how to install it on Windows with one click, and how to use every page of the app. Technical readers should read README.MD and RESOURCES/FORMULAS.MD instead; this file is for everyone else.

In one sentence

Before a risky payment leaves your account, IntentPay asks **why you are paying**, checks your answer for the patterns of a scam, and shows you — in your own language — whether the payment is safe and what it does to your monthly budget.

Version	Date	Audience	Runs on
1.2	30 September 2026	Anyone — no technical background needed	Windows 10 / 11, Python 3.10+

Contents

§	Chapter	What it covers
1	What IntentPay is for	The problem, the idea, who it helps
2	Before you start	What you need on the computer
3	Installing it (one click)	START_HERE.BAT, and what to do if Windows complains
4	The folders and files	Every file, explained in plain words
5	The six pages of the app	What each page is for
6	Checking a payment, step by step	The send-money card and the “why are you paying?” question
7	Needs, wants and unnecessary	The one-line view of your month, in Tamil too
8	“Why this score?”	Reading the reasons behind any warning
9	The UPI black box	A tamper-proof record of what happened
10	Emergency PIN and decoy mode	What to do if someone forces you to pay
11	Local AI on your own computer	Optional: run a real model with Ollama
12	Tamil, elder mode and voice	Using the app in தமிழ்
13	The judge panel	Changing every setting while the app runs
14	If something goes wrong	Common problems and the fix
15	What this prototype does not do	Honest boundaries
16	Questions you may be asked	Short answers
17	Team, licence and credits	Who built it, what you may do with it

Who it is for

Two people, on purpose. **Arun Kumar**, 34, who wants a second opinion before a big transfer. And **Meenakshi Ammal**, 68, who prefers Tamil, larger text and spoken answers, and who wants to see her month in one line. Switch between them in the sidebar and the whole app — language, type size, voice — follows the person.

1. What IntentPay is for

The problem

India processed **185.8 billion UPI payments in a year**. In the same period banks reported **13,516 digital-payment frauds — 56.5 % of all banking frauds, about ₹520 crore**. Surveys in 2025 found that **1 in 5 families** has been hit by a UPI scam, and that **51 % of victims never filed a complaint anywhere**.

The scams are not clever computer attacks. They are phone calls: someone pretending to be your bank, the police, a customer-care agent or a lottery office, talking to you urgently until you send the money yourself. The payment is “authorised” — by you, under pressure — so no bank can stop it afterwards.

The idea

Every safety system asks “*is this payment suspicious?*” IntentPay asks “**why are you paying?**” — before the money leaves. Your own words are the signal: “bank officer said my account will be blocked” and “cousin's college fees” describe the same ₹25,000 payment, but only one of them is a scam in progress.

And because a payment can be perfectly safe and still wreck your month, IntentPay keeps your money legible at the same time: what you spent, what is left, and what you **had** to spend versus what you **chose** to spend.

What it is **not**: it is a hackathon prototype running on synthetic (made-up) data, fully offline. It is not connected to any bank, any UPI app or any real account, and it never asks for a UPI PIN, OTP or password — there is no such field anywhere in the app.

Voice is optional

Spoken answers (Tamil or English) use a free package that installs automatically. If it is missing, the app says so politely and everything still works by typing. Voice never controls a payment — it only reads answers out loud.

2. Before you start

You need	Why	Notes
A Windows 10 or 11 computer	The one-click installer is a Windows batch file	The app itself also runs on macOS/Linux with <code>`python -m streamlit run APP.py`</code>
Python 3.10 or newer	The app is written in Python	Install from python.org and tick “ Add python.exe to PATH ”
About 500 MB of free space	A small virtual environment is created	Most of it is the Python packages
An internet connection, once	Only to download the packages during install	After that the demo runs with no internet at all
Nothing else	No database, no server, no account	Optional voice features install automatically and are never required

3. Installing it (one click)

Step by step

- Install Python 3.10+ from **python.org/downloads**. On the first screen tick “**Add python.exe to PATH**”, then finish the installer.
- Unzip the project so you have a folder called **INTENTPAY** (for example on your Desktop).
- Double-click **START_HERE.BAT**.
- Wait. The black window creates a private environment, installs the packages and then opens your web browser at **http://127.0.0.1:8501**. This takes one to three minutes the first time, and a few seconds every time after that.
- Use the app in the browser. To stop it, close the black window (or press **Ctrl+C** in it).

Nothing is sent anywhere

The app runs entirely on your computer. It never uploads your data, never contacts a bank, and stores nothing outside the project folder. If you turn on the optional external AI engine, the app shows a visible warning that the reason you typed would leave the machine — and it is off by default.

The other files you can double-click

File	What it does
START_HERE.BAT	Install and launch. This is the one you want.
INSTALL.BAT	Install only — use it if you want to set things up before a demo.
RUN.BAT	Launch only — skips the install checks, fastest way to start a second time.

If Windows complains

What you see	What it means	What to do
"Windows protected your PC" (SmartScreen)	The installer files are not signed with a paid certificate	Click More info → Run anyway . The files are plain text — you can read them in Notepad first.
"python is not recognized"	Python was installed without adding it to PATH	Install Python again and tick "Add python.exe to PATH", then reboot.
The browser opens but shows an error	The app is still starting	Wait 5 seconds and refresh the page.
"Port 8501 is already in use"	Another copy of the app is already running	Close the other black window, or open <code>http://127.0.0.1:8502</code> instead.

4. The folders and files

Everything is in ALL CAPS on purpose: the file names are part of the deliverable, and they are easy to spot in a folder listing. Python files keep a lowercase **.py** ending because that is the only ending Python can import.

The top level

File / folder	In plain words
START_HERE.BAT	One-click install and launch.
INSTALL.BAT / RUN.BAT	Install only / launch only.
APP.py	The app itself. Double-clicking it is not enough — it needs Streamlit (<code>python -m streamlit run APP.py</code>), which is why the .BAT files exist.
USER_MANUAL.PDF	This manual.
README.MD	The technical overview: pages, engine, metrics, design boundaries.
AGENTS.MD	The build log: what was decided, what was measured, what the tests caught. Useful if you want to trust the numbers.
REQUIREMENTS.TXT	The list of packages the installer downloads.
REQUIREMENTS_VOICE.TXT	Optional voice packages (never required).
ENV_EXAMPLE.TXT	A template for the optional external AI engine settings.
LICENSE	MIT licence — you may reuse the code with attribution.

The SRC folder — the engine

File	In plain words
CONFIG.py	All settings and their defaults, plus the checks that reject nonsense (for example weights that do not add up to 1).
MODELS.py	The shapes of the data, Indian money formatting (₹1,00,000), distances.
LANGUAGE.py	Every piece of text in the app, in English and Tamil (230 strings).
FEATURES.py	Turns a payment into ~20 numbers: amount, time, distance from home, battery, new recipient, and so on.
ANOMALY.py	Learns what is normal for this customer and scores how unusual a payment is (an Isolation Forest, trained on their own history).
INTENT_ENGINE.py	Reads your reason for paying and scores it against the patterns of real scams — in English and Tamil. An optional online AI can replace it, and always falls back to this local one on failure.
CATEGORIZER.py	Guesses what a payment was for (food, rent, transfer...) and learns from your corrections.
BUDGET.py	What is left of your monthly limit, and the new needs / wants / unnecessary split.

File	In plain words
FORECAST.py	Projects where your spending will land by month end.
RECURRING.py	Finds commitments (rent, subscriptions) and what they leave free.
RISK_ENGINE.py	Combines the three risk scores into one, decides the colour band, and produces the exact “why this score” breakdown.
EXPLAIN.py	Writes the explanations in plain language, and documents how each layer would work in production.
BLACKBOX.py	The tamper-evident event log (see chapter 9).
SIMULATION.py	The payment flow: draft → check → ask why → warn → your decision. Also the emergency PIN and decoy mode.
METRICS.py	Precision, recall, F1 and the cost model used on the Accuracy page.
DATA_GENERATOR.py	Creates the two demo customers, their histories, the 385 labelled test payments and the 10 scenarios — all from fixed seeds, so every run is identical.
DATA_LOADER.py	Reads that data, and remembers your category corrections.
VOICE.py	Optional speech in and out; never fatal if missing.
FRAUD_HELP.py	The honest “I think I was scammed” steps and a complaint draft.
DASHBOARD.py	The flat white building blocks (cards, tables, gauges).

The other folders

Folder	In plain words
DATA/	The generated demo data: two customers' histories, the labelled test set, the scenarios and the profiles. Delete it and it is recreated on the next run.
RESOURCES/	The pitch, the demo script, the formula reference, the architecture diagram and the research sources.
TESTS/	The automated checks: 52 engine tests and 107 UI checks. Run them if you want proof the app behaves (see chapter 13).
.github/	Continuous-integration settings so the tests also run on GitHub.

No dead buttons

Every button in the app does something visible, and the **SAFE DEMO RESET** button in the sidebar puts every setting back to its factory value. If a judge changes something strange, that button undoes it in one click.

5. The six pages of the app

The sidebar on the left moves between them. You can also open a page directly by adding **?page=** to the address, for example `http://127.0.0.1:8501/?page=nav_dashboard`.

Page	What it is for
Payment simulator	The heart of the app. Fill in any payment — amount, method, recipient, time, place, channel, whether you are on a call, battery, device — plus your reason for paying. Press Check this payment and get a score, an explanation, the budget impact and a decision. Ten ready-made scam and normal scenarios are one click away, and the golden demo shows the same ₹25,000 payment with two different reasons.
Money dashboard	Your month: what you spent, what is left, where it will land by month end, the category breakdown, and the new needs / wants / unnecessary split. You can fix a category with one tap.
Judge panel	Every setting, live. Weights, thresholds, limits, the triage map, the emergency PIN and decoy limits, the customer's details, the AI engine, privacy switches, an audit trail and a JSON report. Change something and every page follows immediately.
Accuracy & cost	How well the engine does on 385 labelled payments: precision, recall, F1, false alarms, per-scam-type recall, what each layer adds, and the cost of being wrong. On the shipped settings: precision 0.94, recall 0.83, F1 0.89, false alarms 4.3 % .
Scam help	The honest path after a scam: call 1930 , file at cybercrime.gov.in , dispute with your bank using the UTR, keep the evidence. It also writes a complaint draft you can copy.
About & research	What the app claims, how it is built, the four production layers, the sources behind every claim, and the limitations stated plainly.

6. Checking a payment, step by step

The send-money card

Open **Payment simulator**. The card is headed “**Send money**” and it works like any payment screen you already know: amount, method, recipient name, recipient UPI ID, what the payment is for, when, where, and whether you are on a call right now. Nothing is pre-filled with a verdict — every field is yours to change.

The reason box

On the right is a box for your **reason**, in your own words, in English or Tamil. This is the part no other payment app asks for. It is optional for ordinary payments, and required when the payment looks risky.

What happens when you press “Check this payment”

- The app looks at the payment itself (amount, time, place, device, battery, recipient) and scores it.
- If that first score is **below the ask threshold (55)**, you get your answer immediately — a normal payment is never interrupted. This is deliberate: a safety tool that stops everyone gets ignored by everyone.
- If it is **at or above the threshold**, the app stops and asks “**Why are you paying?**” — and the result card says so, in plain words: “*Reason requested: yes*”.

- You type the reason and press **Check the reason**. The app re-reads the payment with your words included, and the score can move a long way — the golden demo moves it by about **40 points** on the same ₹25,000.
- You then see the verdict, the reasons behind it, what it does to your budget, and the simulated payment handoff.

The app never blocks you — and never lies about it

Budget warnings never stop a payment either: the app says what a payment does to your month and leaves the decision to you. And it says plainly that a risk score is **decision support, not proof of fraud**, and that once money has left, only the bank, NPCI or the recipient can bring it back.

Reading the verdict

Band	Meaning	What the app does
LOW (0–29)	Nothing unusual found	Lets the payment through, no interruption
MEDIUM (30–54)	Something is worth a look	Asks you to review before continuing
HIGH (55–74)	Several scam patterns present	Strong warning + steps to verify
CRITICAL (75–100)	Textbook social-engineering pattern	Tells you to stop, and offers the 1930 / cybercrime.gov.in path

Why this matters for an older user

The card answers one question in one line: *“what did I have to spend, and what did I choose to spend?”* — in the reader's own language, with the amount first and no jargon. Transfers to your own account are never counted as spending, so moving money between your own accounts does not look like an expense.

7. Needs, wants and unnecessary

Most people know what they earn and almost none know what they **chose** to spend. The dashboard now splits your month into three buckets, with the money and the share of each:

Bucket	This month (Arun)	Share	Plain meaning
Needs	₹6,227	33 %	Things you had to pay: rent, food, bills, travel
Wants	₹6,036	64 %	Things you chose: eating out, shopping, entertainment
Unnecessary	₹430	2 %	Duplicates, fees, impulse buys — the easy savings
Total spent	₹12,693	100 %	of a ₹15,000 monthly limit → ₹2,307 left

For the Tamil elder customer the same card is in Tamil and larger:

Bucket	Tamil	This month (Meenakshi Ammal)
Needs	தேவையான சலுகைகள்	the essentials, shown first
Wants	விரும்ப சலுகைகள்	the choices, shown second
Unnecessary	தேவையற்ற சலுகைகள்	the waste, shown last — small and honest

You decide what counts

There is no magic. The app uses a simple table that says which category belongs in which bucket — food and rent are needs, shopping is a want, bank charges are unnecessary. **That table is editable while the app runs:** Judge panel → **Spending triage map** → change any category → **Apply**. The dashboard card follows immediately. Anything the table does not mention is counted as a want, never as an error.

How the layers map to production

The About page shows the four layers a production system would use — a graph neural network for behaviour, graph edges for connections, gradient boosting for the transaction details, a calibrated ensemble for fusion, and SHAP for the explanation — next to what this prototype actually uses, and why. The prototype's own contribution is the intent layer, which is why it is measured separately on the Accuracy page.

8. “Why this score?”

Under every result the app lists the signals it found, biggest first, each with the number of points it added (or removed). This is the same idea researchers call SHAP values: how much each ingredient contributed to the final dish.

What makes it trustworthy is that the numbers **add up exactly**. Add the column and you get the score on the card — the largest gap anywhere in the demo is **0.2 points out of 100**, and usually it is zero. There is no unexplained remainder, so nobody has to take the app's word for it.

A real example (fake bank call, ₹25,000 to a new recipient)

Signal	Points	In plain words
Threats in the reason	+22.5	“your account will be blocked”
Secrecy in the reason	+21.4	“do not tell anyone”
Unusual for this customer	+16.8	nothing like your own history
First payment to this recipient	+5.2	you have never paid them before
On a call right now	+...	the classic setup for coercion
Low battery	+...	scammers keep you on the phone until it dies

Every row appears in **English and Tamil**, so the explanation is never the part of the app that excludes someone.

9. The UPI black box

Aircraft carry a black box so that after an accident the sequence of events can be reconstructed. IntentPay keeps one for every payment: what the app saw, when it asked you something, what you answered, what it decided and what it showed you.

Each entry is sealed to the one before it with a cryptographic fingerprint (SHA-256), so changing or deleting any entry breaks every entry after it. Press **Verify** and the app re-computes the whole chain and tells you it is intact — or exactly which entry was touched.

Try it (it takes twenty seconds)

- Check any payment (the digital-arrest scenario is a good one).
- Scroll to **Safety features** → **UPI black box**. Press **Verify** → the chain is intact.
- Press **Tamper with an entry** → the app edits its own record and the chain breaks at that entry.
- Press **Verify** again → intact.
- Press **Export incident report** → a JSON file with the timeline, the fingerprints and the integrity check. That is what a bank or an investigator would be handed.

Honest note — this part is a concept

There is **no real UPI integration** behind the black box: the events come from the simulator, not from a bank. The chain also lives on your device, so it is **tamper-evidence** for a dispute rather than proof against a compromised phone. The production step is to anchor the fingerprints to a notary or a ledger, which the export already makes possible. The app says all of this on screen.

10. Emergency PIN and decoy mode

The hardest scam to defend against is the one happening to you right now: someone beside you, on the phone, telling you to transfer money *now*. If you cannot say no, you need somewhere to send them.

The emergency PIN opens a **decoy** — a second, restricted environment that looks like the app but is not connected to anything. In the demo it shows a balance of ₹1,240 with a per-transaction limit of ₹500 and a monthly limit of ₹3,000. Payments over the limit are blocked; payments inside it are **simulated**, so the person pressuring you sees a payment go through while nothing real moves. Meanwhile the app quietly records that a duress PIN was used.

Try it

- Scroll to **Safety features** → **Emergency PIN / decoy mode**.
- Type the demo PIN **9111** (you can see and change it in the Judge panel) and press **Activate decoy mode**.
- Try to pay ₹25,000 → blocked, reason *over limit*.
- Pay ₹200 → *simulated*. Nothing real happened.
- Judge panel → set the decoy transaction limit to **0** → now every amount is blocked (*limit zero*).
- Press **Exit decoy mode** to return to the real app. The black box still shows *decoy_activated*, *silent_alert* and every simulated payment.

Honest note — this part is a concept

The PIN, the limits and the simulated payments are real code paths, but they move no real money. A production decoy must be **undiscoverable**: no banner saying “simulation”, no “restore real environment” button, and the PIN never stored in a settings file. This prototype keeps those visible on purpose so that you can see the mechanism working.

Voice never controls money

Speech is for asking questions and hearing answers. It cannot approve, cancel or change a payment, and raw audio is discarded immediately unless you turn recording on yourself.

12. Tamil, elder mode and voice

Tamil is not a translation layer bolted on at the end. It is a first-class language of the app: every label, verdict, explanation and scenario exists in English and Tamil (230 strings), the amount always comes first, sentences are short, and there is no jargon.

The easiest way to see it: in the sidebar, switch the customer to **Meenakshi Ammal — Coimbatore (age 68)**. The whole app turns Tamil, elder mode switches on and the type gets bigger — because those settings belong to the person, not to the app. You can still override the language or elder mode by hand at any time.

Feature	What it does	Why
Tamil UI	Every screen, verdict and explanation	The people most at risk from these scams are often least served in English
Elder mode	Larger text and bigger touch targets	Readable at arm's length
Voice answers	Press “Speak” and ask a question out loud; the answer is spoken back	Lower friction than typing for many older users
Voice is optional	If the packages are missing the app says so and keeps working	A demo must never fail because of an optional feature

Try asking, in either language: *“how much did I spend this month?”* or *“இந்த மாதம் எவ்வளவு செலவாகிறது?”* — the answer includes the total, the budget and what is left.

Two things a judge can check in ten seconds

1) Set the intent weight to 0 and watch the scam scenarios lose their edge — the intent layer is doing real work. 2) Set the “ask why” threshold to 0 and watch the app start interrupting every single payment, including ₹500 for groceries. Both effects are measured on the Accuracy page as well.

13. The judge panel

This page exists because a prototype that only accepts one input is a slide, not a prototype. Everything below can be changed while the app is running, and every other page follows immediately. Invalid input is rejected with the exact reason, and the last good setting is kept — the app never crashes because of a typo.

Group	What you can change
Fusion weights	How much the transaction, the reason and the recipient each count (they must add up to 1 — try entering 0.5 / 0.5 / 0.5 and you get a clear error)
Thresholds	The three band edges and the “ask why” score (default 55)
Anomaly layer	Blend, contamination, number of trees — watch the scores move
Budget	Monthly limit, food limit, forecast window, and the needs / wants / unnecessary triage map
Emergency PIN	The PIN itself, the decoy balance, the per-transaction limit and the monthly limit
Customer	Name, age, city, limits, known devices, and the history seed — a different seed generates a completely different demo customer
AI engine	Local ⇌ external (with no key set, the fallback is visible)
Privacy	Local-only switch, raw-voice recording, audit trail, JSON report export
Reset	SAFE DEMO RESET puts every setting back to its factory value

Where the build log lives

AGENTS.MD records every decision, every measured number and every bug the tests caught, including the ten issues found by driving the real app. If a number in this manual is ever doubted, that file is where it came from.

14. If something goes wrong

Symptom	Likely cause	Fix
The browser shows “Connection refused”	The app has not finished starting	Wait five seconds and refresh; or run RUN.BAT again
A page is blank or shows a red error	A setting was pushed out of range	Press SAFE DEMO RESET in the sidebar
Voice buttons do nothing	The optional voice packages are not installed	Type instead — everything works by typing; or install REQUIREMENTS_VOICE.TXT
Numbers look odd after editing the customer	A limit was set to zero	Set a sensible limit, or SAFE DEMO RESET
The window closed itself	Ctrl+C was pressed, or the laptop slept	Double-click START_HERE.BAT again — installs are remembered

Proving it still works

If you want to check the app end to end, open a command prompt in the project folder and run the two test files. They need no extra setup:

Command	What it checks	Current result
python TESTS\RUN_TESTS.py	52 engine checks: money maths, categories, budgets, forecasts, scam detection in English and Tamil, the triage split, the black box chain, the decoy limits, and that the attribution adds up	52 / 52 pass
python TESTS\RUN_UI_TESTS.py	107 checks that drive the real app in a browser: every page renders, every button works, the reason prompt appears only when it should, the tamper demo breaks the chain, the decoy limit blocks	107 / 107 pass

15. What this prototype does not do

Saying this out loud is part of the design. A safety tool that overclaims is worse than one that admits its edges.

- It runs on **synthetic data**. That proves the pipeline works, not that the accuracy numbers would hold on real customers.
- It is **not connected to any bank or UPI app**. There is no live account, no real payment and no statement import.
- It **never asks for a UPI PIN, OTP, password or card number** — there is no such field anywhere in the app, and there never will be.
- It **cannot reverse a payment** that has already left. Only the bank, NPCI or the recipient can. That is why the scam-help page is one click from every risky result.
- A risk score is **decision support, not proof of fraud**, and a budget warning is guidance, not financial advice. Neither blocks a payment by itself.
- The **UPI black box** and the **decoy mode** are concepts: see chapters 9 and 10 for exactly what is simulated and what production would have to add.

- There is **no covert monitoring**: no call recording, no WhatsApp scraping, no reading of other apps. The reason you type stays on your machine unless you switch the external AI engine on yourself, which the app warns about on screen.
- “IntentPay” is a project codename for this hackathon — rename it before any commercial use.

16. Questions you may be asked

Question	Short answer
Why not just use Google Pay?	Spending insights already exist in ten Indian languages. Nobody asks why you are paying before a risky transfer, and nobody connects that answer to your budget in the same moment.
How do you get bank data?	Synthetic demo data plus CSV import in the prototype. A regulated, consent-based integration is the production path — we never claim live bank access.
Can you reverse a UPI payment?	No, and the app says so. It routes you to 1930, cybercrime.gov.in and your bank inside the golden hour.
Isn't this just a rule engine?	The anomaly layer learns each customer's own baseline; the intent layer is a weighted bilingual model; both are ablated and measured. Rules alone score $F1 \approx 0$ at the default threshold — deliberately conservative.
How accurate is it?	On 385 labelled synthetic payments: precision 0.94, recall 0.83, F1 0.89, false alarms 4.3 %, MCC 0.80. Perfect recall on bank impersonation, digital arrest and lottery scams; 64–75 % on the genuinely hard ones (refund QR, fake support, family emergency, job fees).
How do you avoid crying wolf?	The reason is only asked when the payment already looks risky, and the cost model on the Accuracy page prices a false alarm against a missed scam.
What is genuinely new?	Conditional, explainable user-stated intent as a first-class fraud signal, fused with budget impact in one pre-payment moment — plus the needs/wants view for older users, exact attribution, a tamper-evident black box and a duress-PIN decoy.
What does it cost to run?	About 13 ms per decision on an ordinary laptop once the model is cached, and roughly 0.5 s for the first decision after a history change.

One last thing

If you only press three buttons in this app, press these: **Fake bank call**, then **Why this score?**, then **SAFE DEMO RESET**. That is the whole idea — the app reads the reason you were about to give a stranger, explains itself, and hands the decision back to you.

17. Team, licence and credits

Item	Detail
Project	IntentPay — AI Payment Intent Guardian + Personal Financial Guardian
Team	NAS CODERS
Event	CitiBank-organised fintech hackathon (prototype track)
Version / date	1.1 · 30 September 2026
Licence	MIT — reuse the code with attribution (see LICENSE)
Data	All synthetic and generated from fixed seeds; no real customer data exists
Fonts	Body text: Helvetica. Tamil text: Noto Sans Tamil, SIL Open Font License 1.1

Research behind the claims

The numbers quoted in this manual come from the RBI Annual Report 2024-25 (13,516 digital-payment frauds, 56.5 % of banking frauds, ₹520 crore; 185.8 bn UPI transactions), I4C and NPCI digital-arrest advisories, a 2025 LocalCircles survey (1 in 5 families, 51 % never complained), NPCI AutoPay circular OC-223, and published work on hybrid machine learning for UPI fraud and on cost-sensitive evaluation (Elkan 2001, Bahnsen et al. 2015). The full list with links is in **RESOURCES/RESEARCH.MD** and on the About page.